

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 5

AGGREGATE AND GROUPING QUERIES

1. Aim

To summarise data using aggregate functions with GROUP BY and HAVING, and to generate subtotals using ROLLUP and CUBE (emulated where MySQL does not support it).

2. Problem Statement

The examination cell of the college needs summary reports from *college_db*:

1. Overall statistics of all enrollments and the number of students having a city.
2. Course-wise number of students, average, highest and lowest marks.
3. Department-wise and semester-wise number of enrollments and average marks.
4. Courses with an average above 75, departments with at least 3 students, and students scoring 70 or more in at least two courses.
5. The list of courses and total credits of every student.
6. Salary bill of instructors with department subtotals and a grand total, and an enrollment summary with subtotals (ROLLUP) and all cross-totals (CUBE).

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To use the aggregate functions COUNT, SUM, AVG, MIN, MAX and GROUP_CONCAT.
2. To group rows using GROUP BY on one or more columns.
3. To filter groups using HAVING and distinguish it from WHERE.
4. To produce subtotals and grand totals using WITH ROLLUP and GROUPING().
5. To understand CUBE and emulate it in MySQL.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Aggregate Functions

An aggregate function computes a single value from a set of rows. Except COUNT(*), aggregate functions ignore NULL values.

Function	Returns	NULL handling
COUNT(*)	Number of rows	Counts every row
COUNT(col) / COUNT(DISTINCT col)	Number of non-NULL (distinct) values	Ignores NULL
SUM(col)	Total of the values	Ignores NULL; NULL if no rows
AVG(col)	Arithmetic mean = SUM / COUNT(col)	Ignores NULL
MIN(col), MAX(col)	Smallest / largest value (numbers, strings, dates)	Ignores NULL
GROUP_CONCAT(col)	Values of the group joined into one string (MySQL-specific)	Ignores NULL

5.2 GROUP BY and HAVING

```
SELECT    grouping_columns, aggregate_function(column)
FROM      table
[WHERE    row_condition]
GROUP BY  grouping_columns
[HAVING   group_condition]
[ORDER BY ...];
```

GROUP BY divides the rows into groups having equal values in the grouping columns and returns one row per group. The clauses are evaluated in the order FROM - WHERE - GROUP BY - aggregate computation - HAVING - SELECT - ORDER BY. Hence aggregate functions cannot be used in WHERE; a condition on an aggregate must be placed in HAVING.

5.3 ROLLUP, CUBE and GROUPING SETS

These extensions compute several levels of aggregation (super-aggregates) in one query, as needed for reports with subtotals:

- **ROLLUP (a, b)** produces the groupings (a, b), (a) and () - i.e. subtotals for each *a* and a grand total. MySQL syntax: **GROUP BY a, b WITH ROLLUP**.
- **CUBE (a, b)** produces all 2^n combinations: (a, b), (a), (b) and ().
- **GROUPING SETS ((a), (b))** produces exactly the listed groupings.
- **GROUPING(col)** returns 1 when the NULL in a super-aggregate row was produced by ROLLUP and 0 for a normal row. It is available in MySQL from version 8.0.1.

Feature	MySQL 8.0	PostgreSQL / SQL Server / Oracle
ROLLUP	GROUP BY a, b WITH ROLLUP	GROUP BY ROLLUP (a, b)
CUBE	Not supported - emulate with UNION ALL	GROUP BY CUBE (a, b)
GROUPING SETS	Not supported - emulate with UNION ALL	GROUP BY GROUPING SETS (...)
GROUPING()	Supported (8.0.1+)	Supported

6. Procedure / Algorithm

1. Create and populate *college_db* (Q1, Q2).
2. Apply COUNT, SUM, AVG, MIN and MAX to the whole enrollment table and compare COUNT(*) with COUNT(column) on a column containing NULL.
3. Group the enrollments by course and compute the statistics of each course.
4. Group by two columns after joining enrollment, course and department.
5. Filter groups using HAVING and combine WHERE and HAVING in one query.
6. Use GROUP_CONCAT to list the courses of each student.
7. Use WITH ROLLUP to obtain subtotals and a grand total; label the super-aggregate rows using GROUPING().
8. Attempt GROUP BY CUBE, note that MySQL rejects it, and emulate CUBE with UNION ALL.
9. Observe the ONLY_FULL_GROUP_BY error and record all outputs.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```

DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2) DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
    instr_id      INT          NOT NULL,
    instr_name    VARCHAR(40)  NOT NULL,
    designation   VARCHAR(20)  NOT NULL,
    dept_id      INT,
    salary        DECIMAL(10,2) NOT NULL,
    email         VARCHAR(40)  NOT NULL,
    mentor_id     INT,
    CONSTRAINT pk_instructor  PRIMARY KEY (instr_id),
    CONSTRAINT uq_instr_email UNIQUE (email),

```

```

CONSTRAINT chk_salary      CHECK (salary BETWEEN 30000 AND 300000),
CONSTRAINT fk_instr_dept   FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL
);

CREATE TABLE student (
    student_id INT NOT NULL AUTO_INCREMENT,
    roll_no    VARCHAR(10) NOT NULL,
    student_name VARCHAR(40) NOT NULL,
    gender     CHAR(1) NOT NULL,
    dob        DATE NOT NULL,
    dept_id    INT NOT NULL,
    city       VARCHAR(30) DEFAULT 'Pune',
    email      VARCHAR(40) NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no  UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender  CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob     CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id INT NOT NULL,
    phone      CHAR(10) NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone        CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id CHAR(5) NOT NULL,
    title     VARCHAR(40) NOT NULL,
    dept_id   INT NOT NULL,
    credits   TINYINT NOT NULL,
    instr_id  INT,
    prereq_id CHAR(5),
    CONSTRAINT pk_course PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title UNIQUE (title),
    CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
    REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id INT NOT NULL,
    course_id  CHAR(5) NOT NULL,
    semester   TINYINT NOT NULL,
    marks      INT,
    grade      CHAR(2),
    CONSTRAINT pk_enrollment PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8),
    CONSTRAINT chk_marks    CHECK (marks BETWEEN 0 AND 100),
    CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enr_course FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL

server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
  (10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
  (20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
  (30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
  (40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
  (50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
  (101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
  (102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
  (103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
  (104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
  (105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
  (106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
  (107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
  (108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
  (roll_no, student_name, gender, dob, dept_id, city, email) VALUES
  ('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
  ('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
  ('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
  ('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
  ('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
  ('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
  ('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
  ('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
  ('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
  ('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
  (1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
  (4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
  ('CS101', 'Programming in C', 10, 4, 103, NULL),
  ('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
  ('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
  ('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
  ('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
  ('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
  ('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
  ('EC201', 'Digital Electronics', 30, 3, 105, NULL),
  ('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
  ('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
  (1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
  (1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
  (2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
  (3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
  (4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
  (5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
  (6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),
  (8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
  (9, 'CS401', 4, 88, 'A+');

```

Q3: Aggregate functions over a whole table; COUNT(*) versus COUNT(column)

```

SELECT COUNT(*)           AS enrollments,
       COUNT(DISTINCT student_id) AS students,
       SUM(marks)          AS total_marks,
       ROUND(AVG(marks), 2) AS avg_marks,
       MIN(marks)          AS min_marks,
       MAX(marks)          AS max_marks
FROM   enrollment;

SELECT COUNT(*) AS all_students, COUNT(city) AS with_city,
       COUNT(DISTINCT city) AS distinct_cities
FROM   student;

```

Q4: GROUP BY one column: course-wise result analysis

```

SELECT course_id,
       COUNT(*)           AS students,
       ROUND(AVG(marks), 2) AS avg_marks,
       MAX(marks)          AS highest,
       MIN(marks)          AS lowest
FROM   enrollment
GROUP BY course_id
ORDER BY avg_marks DESC;

```

Q5: GROUP BY with a join and two grouping columns

```

SELECT d.dept_code, e.semester,
       COUNT(*) AS enrollments, ROUND(AVG(e.marks), 2) AS avg_marks
FROM   enrollment e
JOIN   course c     ON c.course_id = e.course_id
JOIN   department d ON d.dept_id = c.dept_id
GROUP BY d.dept_code, e.semester
ORDER BY d.dept_code, e.semester;

```

Q6: HAVING: filtering groups (and WHERE together with HAVING)

```

SELECT course_id, ROUND(AVG(marks), 2) AS avg_marks
FROM   enrollment
GROUP BY course_id
HAVING AVG(marks) > 75
ORDER BY avg_marks DESC;

SELECT dept_id, COUNT(*) AS students
FROM   student
GROUP BY dept_id
HAVING COUNT(*) >= 3;

SELECT student_id, COUNT(*) AS courses_passed_with_A_or_better
FROM   enrollment
WHERE  marks >= 70
GROUP BY student_id
HAVING COUNT(*) >= 2
ORDER BY student_id;

```

WHERE removes individual rows *before* grouping (marks below 70 are not counted), while HAVING removes whole groups *after* the aggregates are computed.

Q7: GROUP_CONCAT: list the courses of each student in one row

```

SELECT s.roll_no,
       GROUP_CONCAT(e.course_id ORDER BY e.course_id SEPARATOR ', ') AS courses,
       SUM(c.credits) AS total_credits
FROM   student s
JOIN   enrollment e ON e.student_id = s.student_id
JOIN   course c     ON c.course_id = e.course_id
GROUP BY s.roll_no
ORDER BY s.roll_no;

```

Q8: WITH ROLLUP: subtotals and grand total, labelled using GROUPING()

```

SELECT d.dept_code, i.designation,
       COUNT(*) AS staff, SUM(i.salary) AS total_salary
FROM   instructor i
JOIN   department d ON d.dept_id = i.dept_id
GROUP BY d.dept_code, i.designation WITH ROLLUP;

SELECT IF(GROUPING(d.dept_code), 'ALL DEPTS', d.dept_code) AS dept,
       IF(GROUPING(e.semester), 'All sem', CONCAT('Sem ', e.semester)) AS sem,
       COUNT(*) AS enrollments, ROUND(AVG(e.marks), 2) AS avg_marks
FROM   enrollment e
JOIN   course c     ON c.course_id = e.course_id
JOIN   department d ON d.dept_id = c.dept_id
GROUP BY d.dept_code, e.semester WITH ROLLUP;

```

Q9: CUBE: not supported by MySQL 8.0, emulated with ROLLUP and UNION ALL

```

SELECT d.dept_code, e.semester, COUNT(*) AS enrollments
FROM   enrollment e JOIN course c ON c.course_id = e.course_id
JOIN   department d ON d.dept_id = c.dept_id
GROUP BY CUBE (d.dept_code, e.semester);

SELECT d.dept_code, e.semester, COUNT(*) AS enrollments
FROM   enrollment e JOIN course c ON c.course_id = e.course_id
JOIN   department d ON d.dept_id = c.dept_id
GROUP BY d.dept_code, e.semester WITH ROLLUP
UNION ALL
SELECT NULL, e.semester, COUNT(*)
FROM   enrollment e
GROUP BY e.semester;

```

ROLLUP (dept_code, semester) produces the groupings (dept, sem), (dept) and (); CUBE also needs the grouping (sem). The second SELECT adds the semester-wise subtotals, so the union contains all $2^2 = 4$ groupings of a CUBE. The NULLs here are grouping NULLs, because neither column contains real NULL values.

Q10: ONLY_FULL_GROUP_BY: a non-aggregated column that is not in GROUP BY

```

SELECT dept_id, student_name, COUNT(*) FROM student GROUP BY dept_id;
SELECT @@sql_mode\G

```

Under the default SQL mode ONLY_FULL_GROUP_BY, every selected column must either appear in GROUP BY or be inside an aggregate function, because one group has many student names.

8. Sample Input / Database Tables

The following tables of *college_db* (created by Q1 and Q2) are used. The values were read from the database after executing the setup queries.

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.02 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.01 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.00 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Aggregate functions over a whole table; COUNT(*) versus COUNT(column)

```
mysql> SELECT COUNT(*) AS enrollments,
-> COUNT(DISTINCT student_id) AS students,
-> SUM(marks) AS total_marks,
-> ROUND(AVG(marks), 2) AS avg_marks,
-> MIN(marks) AS min_marks,
-> MAX(marks) AS max_marks
-> FROM enrollment;
```

enrollments	students	total_marks	avg_marks	min_marks	max_marks
25	9	1906	76.24	45	95

1 row in set (0.00 sec)

```
mysql> SELECT COUNT(*) AS all_students, COUNT(city) AS with_city,
-> COUNT(DISTINCT city) AS distinct_cities
-> FROM student;
```

all_students	with_city	distinct_cities
10	9	4

1 row in set (0.00 sec)

Output of Q4: GROUP BY one column: course-wise result analysis

```
mysql> SELECT course_id,
-> COUNT(*) AS students,
-> ROUND(AVG(marks), 2) AS avg_marks,
-> MAX(marks) AS highest,
-> MIN(marks) AS lowest
-> FROM enrollment
-> GROUP BY course_id
-> ORDER BY avg_marks DESC;
```

course_id	students	avg_marks	highest	lowest
CS401	1	88.00	88	88
AI401	1	84.00	84	84
CS301	4	82.50	95	70
CS201	3	80.67	90	67
AI301	3	76.00	89	63
AI201	4	75.75	92	58
CS101	5	72.80	88	55
EC301	2	71.50	82	61
EC201	2	62.00	79	45

9 rows in set (0.00 sec)

Output of Q5: GROUP BY with a join and two grouping columns

```
mysql> SELECT d.dept_code, e.semester,
->      COUNT(*) AS enrollments, ROUND(AVG(e.marks), 2) AS avg_marks
-> FROM   enrollment e
-> JOIN   course c      ON c.course_id = e.course_id
-> JOIN   department d ON d.dept_id = c.dept_id
-> GROUP BY d.dept_code, e.semester
-> ORDER BY d.dept_code, e.semester;
```

dept_code	semester	enrollments	avg_marks
AIML	3	4	75.75
AIML	4	4	78.00
CSE	3	8	75.75
CSE	4	5	83.60
ECE	3	2	62.00
ECE	4	2	71.50

6 rows in set (0.00 sec)

Output of Q6: HAVING: filtering groups (and WHERE together with HAVING)

```
mysql> SELECT course_id, ROUND(AVG(marks), 2) AS avg_marks
-> FROM   enrollment
-> GROUP BY course_id
-> HAVING AVG(marks) > 75
-> ORDER BY avg_marks DESC;
```

course_id	avg_marks
CS401	88.00
AI401	84.00
CS301	82.50
CS201	80.67
AI301	76.00
AI201	75.75

6 rows in set (0.00 sec)

```
mysql> SELECT dept_id, COUNT(*) AS students
-> FROM   student
-> GROUP BY dept_id
-> HAVING COUNT(*) >= 3;
```

dept_id	students
10	3
20	4

2 rows in set (0.01 sec)

```
mysql> SELECT student_id, COUNT(*) AS courses_passed_with_A_or_better
-> FROM   enrollment
-> WHERE  marks >= 70
-> GROUP BY student_id
-> HAVING COUNT(*) >= 2
-> ORDER BY student_id;
```

student_id	courses_passed_with_A_or_better
1	4
2	2
3	3
4	3
6	2
9	3

6 rows in set (0.00 sec)

Output of Q7: GROUP_CONCAT: list the courses of each student in one row

```
mysql> SELECT s.roll_no,
->         GROUP_CONCAT(e.course_id ORDER BY e.course_id SEPARATOR ', ') AS courses,
->         SUM(c.credits) AS total_credits
-> FROM     student s
-> JOIN     enrollment e ON e.student_id = s.student_id
-> JOIN     course c     ON c.course_id = e.course_id
-> GROUP BY s.roll_no
-> ORDER BY s.roll_no;
```

roll_no	courses	total_credits
S24AI001	AI201, AI301, CS101, CS301	14
S24AI002	AI201, AI301, AI401	10
S24AI003	AI201, AI301	7
S24CS001	AI201, CS101, CS201, CS301	14
S24CS002	CS101, CS201, CS301	11
S24CS003	CS201, CS301, CS401	10
S24EC001	CS101, EC201, EC301	11
S24EC002	EC201, EC301	7
S24ME001	CS101	4

9 rows in set (0.00 sec)

Output of Q8: WITH ROLLUP: subtotals and grand total, labelled using GROUPING()

```
mysql> SELECT d.dept_code, i.designation,
->         COUNT(*) AS staff, SUM(i.salary) AS total_salary
-> FROM     instructor i
-> JOIN     department d ON d.dept_id = i.dept_id
-> GROUP BY d.dept_code, i.designation WITH ROLLUP;
```

dept_code	designation	staff	total_salary
AIML	Assoc. Prof	1	115000.00
AIML	Asst. Prof	1	78000.00
AIML	Professor	1	172000.00
AIML	NULL	3	365000.00
CIVIL	Asst. Prof	1	76000.00
CIVIL	NULL	1	76000.00
CSE	Asst. Prof	1	82000.00
CSE	Professor	1	185000.00
CSE	NULL	2	267000.00
ECE	Professor	1	168000.00
ECE	NULL	1	168000.00
MECH	Assoc. Prof	1	112000.00
MECH	NULL	1	112000.00
NULL	NULL	8	988000.00

14 rows in set (0.00 sec)

```
mysql> SELECT IF(GROUPING(d.dept_code), 'ALL DEPTS', d.dept_code) AS dept,
->         IF(GROUPING(e.semester), 'All sem', CONCAT('Sem ', e.semester)) AS sem,
->         COUNT(*) AS enrollments, ROUND(AVG(e.marks), 2) AS avg_marks
-> FROM     enrollment e
-> JOIN     course c     ON c.course_id = e.course_id
-> JOIN     department d ON d.dept_id = c.dept_id
-> GROUP BY d.dept_code, e.semester WITH ROLLUP;
```

dept	sem	enrollments	avg_marks
AIML	Sem 3	4	75.75
AIML	Sem 4	4	78.00
AIML	All sem	8	76.88
CSE	Sem 3	8	75.75
CSE	Sem 4	5	83.60
CSE	All sem	13	78.77
ECE	Sem 3	2	62.00
ECE	Sem 4	2	71.50
ECE	All sem	4	66.75

```
| ALL DEPTS | All sem |          25 |          76.24 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

Output of Q9: CUBE: not supported by MySQL 8.0, emulated with ROLLUP and UNION ALL

```
mysql> SELECT d.dept_code, e.semester, COUNT(*) AS enrollments
-> FROM enrollment e JOIN course c ON c.course_id = e.course_id
-> JOIN department d ON d.dept_id = c.dept_id
-> GROUP BY CUBE (d.dept_code, e.semester);
ERROR 1064 (42000) at line 219: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'CUBE
(d.dept_code, e.semester)' at line 4

mysql> SELECT d.dept_code, e.semester, COUNT(*) AS enrollments
-> FROM enrollment e JOIN course c ON c.course_id = e.course_id
-> JOIN department d ON d.dept_id = c.dept_id
-> GROUP BY d.dept_code, e.semester WITH ROLLUP
-> UNION ALL
-> SELECT NULL, e.semester, COUNT(*)
-> FROM enrollment e
-> GROUP BY e.semester;
+-----+-----+-----+
| dept_code | semester | enrollments |
+-----+-----+-----+
| AIML      |          3 |            4 |
| AIML      |          4 |            4 |
| AIML      |        NULL |            8 |
| CSE       |          3 |            8 |
| CSE       |          4 |            5 |
| CSE       |        NULL |           13 |
| ECE       |          3 |            2 |
| ECE       |          4 |            2 |
| ECE       |        NULL |            4 |
| NULL      |        NULL |           25 |
| NULL      |          3 |           14 |
| NULL      |          4 |           11 |
+-----+-----+-----+
12 rows in set (0.00 sec)
```

Output of Q10: ONLY_FULL_GROUP_BY: a non-aggregated column that is not in GROUP BY

```
mysql> SELECT dept_id, student_name, COUNT(*) FROM student GROUP BY dept_id;
ERROR 1055 (42000) at line 233: Expression #2 of SELECT list is not in GROUP BY clause and
contains nonaggregated column 'college_db.student.student_name' which is not functionally
dependent on columns in GROUP BY clause; this is incompatible with
sql_mode=only_full_group_by

mysql> SELECT @@sql_mode\G
***** 1. row *****
@@sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVI
SION_BY_ZERO,NO_ENGINE_SUBSTITUTION
1 row in set (0.00 sec)
```

10. Result

Aggregate queries were executed successfully: the 25 enrollments of 9 students have an average of 76.24 marks (range 45-95); course-wise, department-wise and semester-wise summaries were obtained using GROUP BY; HAVING returned only the qualifying groups; WITH ROLLUP produced department subtotals and grand totals, labelled using GROUPING(); and GROUP BY CUBE, which MySQL 8.0 rejects with a syntax error, was emulated using WITH ROLLUP and UNION ALL.

11. Conclusion

Aggregate functions together with GROUP BY convert detailed rows into summary information required for reports. HAVING filters groups, whereas WHERE filters rows before grouping. ROLLUP is a convenient way of producing subtotals and grand totals in a single query. Because MySQL 8.0 implements only ROLLUP, CUBE and GROUPING SETS must be written as a UNION of groupings, while PostgreSQL, SQL Server and Oracle support them directly. The ONLY_FULL_GROUP_BY mode protects against ambiguous queries that select non-grouped columns.